Laboratory 2: LDPC analysis & comparison

MATTEO CAMPAGNARO 2155351

ABSTRACT This experiment investigates the performance of LDPC codes and the Cascade protocol. Two types of channels are considered: a Binary Symmetric Channel (BSC) with a fixed quantum bit error rate (QBER) and channels with a deterministic number of errors at random positions. The study first examines the impact of puncturing and shortening on LDPC codes using a dedicated Python library. Subsequently, LDPC codes are compared with a constructed Cascade protocol to evaluate their error correction capabilities and efficiency. Performance is assessed using two metrics: f_1, the ratio of disclosed bits to the theoretical minimum, and f_2, the difference between disclosed bits and the theoretical lower bound, across varying error rates.

I. INTRODUCTION

The objective of this experiment is to perform a comparative analysis of the performance of LDPC codes and the Cascade protocol in communication scenarios involving two distinct types of channels:

- Binary Symmetric Channel (BSC) with a fixed QBER.
- Channels with a deterministic number of errors occurring at random positions.

In the first part of the experiment, we use a pre-defined LDPC code implemented through a dedicated Python library [Section: (VI)] to study the effect of puncturing and shortening on the protocol's performance.

In the second part, we compare the LDPC code with a constructed Cascade protocol [Section (VII)] to evaluate their performances over the two channels of interest. The goal is to assess the error correction capabilities and efficiencies of the codes using analytical implementations, based on two primary metrics:

- f_1: the ratio of the number of disclosed bits to the theoretical lower bound.
- f_2: the difference between the number of disclosed bits and the theoretical lower bound.

Additionally, the tests are conducted over a range of error rates to evaluate the protocols' behavior under different conditions.

II. INFORMATION RECONCILIATION

Information reconciliation is a fundamental post-processing step in distributed information processing and quantum key distribution (QKD). Its goal is to enable two legitimate parties, to transform correlated but non-identical strings into identical copies while revealing as little information as possible to any external observer. These discrepancies typically arise from noise in the communication channel or measurement imperfections.

Information reconciliation protocols use authenticated public communication to correct these discrepancies. The amount of information leaked during this process must be carefully accounted for, as it directly reduces the secrecy of the final shared data.

A. THEORETICAL LOWER BOUND ON INFORMATION LEAKAGE

The minimum amount of information that must be revealed over the public channel is governed by information theory and is independent of the specific reconciliation protocol used.

Let X and Y denote the random variables corresponding to Alice's and Bob's raw bit strings, respectively. The minimum number of bits that Alice must reveal during reconciliation with Bob is lower bounded by the conditional entropy $H(X|Y)$, which quantifies Bob's uncertainty about Alice's string prior to reconciliation. Any protocol that reveals fewer than $H(X|Y)$ bits cannot, in general, enable reliable error correction.

For a binary symmetric channel (BSC) with crossover probability p, the conditional entropy per bit is given by

$$H(X|Y) = h(p),$$

where $h(p)$ is the binary entropy function. For a block of length n, the theoretical minimum information leakage is therefore

$$\lambda_{\min} = n\, h(p).$$

In practice, real reconciliation protocols necessarily reveal more information than this bound due to finite-length effects,

decoding inefficiencies, and protocol overhead. The efficiency of an information reconciliation protocol is commonly characterized by the reconciliation efficiency parameter. In this work we are going to evaluate:

$$\begin{aligned} f_1 &:= \lambda_{\mathrm{EC}} - n\,h(p) \\ f_2 &:= \frac{\lambda_{\mathrm{EC}}}{n\,h(p)} \end{aligned} \tag{1}$$

where λ_{EC} denotes the actual number of bits revealed during error correction. An ideal protocol achieves $f_1 = 1$ and $f_2 = 0$, while practical implementations typically presents greater f_1 and f_2.

This lower bound plays a central role in security analyses. Consequently, the design of reconciliation protocols aims to approach this bound as closely as possible while maintaining robustness and computational efficiency.

B. INFORMATION-THEORETIC SYMMETRY

Information-theoretic, symmetry refers to the property that an information reconciliation protocol does not intrinsically privilege one party as a noiseless or authoritative reference. Instead, both parties are treated as holding noisy but correlated versions of the same underlying random variable. The protocol can therefore be executed with either party acting as the sender of reconciliation information without affecting its theoretical performance or security.

LDPC-based reconciliation provides a canonical example of role symmetry. Although it is typically implemented as a one-way protocol in which one party transmits a syndrome to the other, either party may assume the role of syndrome sender. This distinguishes theoretic symmetry from communication symmetry, which concerns the directionality and interactivity of message exchange as in the Cascade protocol.

III. TESTED PROTOCOLS

In this section we are going to survey the information reconciliation protocols that has been implemented, tested and compared in this work. The two protocol chosen for evaluation are Low-Density Parity-Check Codes (LDPC) and Cascade.

A. LOW-DENSITY PARITY-CHECK CODES

Low-Density Parity-Check (LDPC) codes are a class of linear block error-correcting codes characterized by a sparse parity-check matrix $\mathbf{H} \in \{0,1\}^{m\times n}$. A binary LDPC code of length n and dimension k is defined as the null space of $\mathbf{H}$, with code rate $R = k/n = 1 - m/n$.

The sparsity of $\mathbf{H}$ enables efficient iterative decoding using message-passing algorithms on a bipartite Tanner graph. In this graph, variable nodes represent codeword bits, while check nodes represent parity constraints imposed by the rows of $\mathbf{H}$. An edge exists between a variable node and a check node if the corresponding entry in $\mathbf{H}$ is nonzero.

Decoding is typically performed using belief propagation (BP) in the log-likelihood ratio (LLR) domain. Each variable node maintains an LLR that reflects the likelihood of the corresponding bit being zero or one, based on both the channel observations and messages received from connected check nodes. Check nodes update outgoing messages by enforcing parity constraints, combining incoming messages from neighboring variable nodes. This iterative exchange continues until either all parity checks are satisfied or a maximum number of iterations is reached.

LDPC flexibility allows the use of techniques such as shortening and puncturing to adapt the effective code rate to varying channel conditions.

a: Shortening.

Shortening is a technique used to reduce the length of a codeword by fixing certain information bits to known values (typically zero) before encoding. This effectively reduces both the code length and the dimension, allowing the code to operate at a lower rate while preserving its error-correcting capabilities. Shortening is particularly useful when the channel conditions are good, as it can improve the code's efficiency by transmitting fewer bits.

b: Puncturing.

Puncturing, on the other hand, increases the code rate by deliberately omitting (or "puncturing") certain encoded bits before transmission. The decoder treats the missing bits as erasures and attempts to recover them using the remaining received bits. Puncturing is valuable in adapting a code to a higher-rate requirement without redesigning the entire code, allowing flexibility in scenarios with varying channel quality.

B. CASCADE PROTOCOL

The Cascade protocol is an interactive, symmetric information reconciliation protocol based on iterative error detection and correction using parity comparisons. It operates by shuffling the raw key and divide it into blocks and publicly comparing the parity of corresponding blocks. When a parity mismatch is detected, a binary search is performed within the block to locate and correct the erroneous bit.

Cascade proceeds in multiple passes. In each subsequent pass, the block size is increased and the bits are randomly permuted to ensure that errors missed or introduced in earlier passes are likely to be detected later.

While Cascade is conceptually simple and robust, it requires many rounds of interactive communication, which lead to a total information leakage higher than that of modern code-based approaches, particularly at low error rates.

IV. EXPERIMENTAL SETUP FOR LDPC AND CASCADE PROTOCOL SIMULATIONS

To evaluate the performance of information reconciliation protocols, we designed a simulation framework for both the LDPCvprotocol and the Cascade protocol. The experimental setup is structured to systematically assess the measurement metrics across different parameters and channel conditions.

1) LDPC Protocol Simulation

The simulation begins by generating a random key sequence whose length matches that of the chosen LDPC code. For computational efficiency, we selected the LDPC code of length 1944 bits. The generated key is then passed through a binary symmetric channel (BSC) or a fixed error channel to introduce controlled noise, simulating realistic transmission errors.

Following the introduction of errors, the LDPC code is applied to the noisy key. Our implementation includes automatic code detection within a predefined code range, allowing the protocol to adaptively select the most suitable code. To further optimize performance, puncturing and shortening techniques are applied as required.

The main performance metrics are evaluated over a quantum bit error rate (QBER) range from 0.01 to 0.2, using a step size of 0.02. In a subsequent analysis, the code rate is fixed at 0.75 to study the effect of puncturing and shortening independently. Specifically, the number of punctured bits is varied among 50, 100, 150, and 200 while keeping shortening fixed at zero, and vice versa. Each configuration is repeated for five iterations to improve the accuracy of the estimated metrics. The initial efficiency is set to 1 for the evaluation of puncturing and shortening effects.

2) Cascade Protocol Simulation

The Cascade protocol simulation follows a similar key generation procedure as the LDPC setup over the same key size: 1944 bits. Due to the protocol's lower computational complexity, we perform 100 iterations for each configuration to obtain statistically significant results. The protocol is evaluated over the same QBER range (0.01 to 0.2) with identical step size.

Cascade employs a multi-pass block-checking strategy. We use seven different block sizes, with the first block size determined as $\frac{0.73}{\text{QBER}}$ to increase the likelihood of detecting errors in the initial pass since if we used $\frac{1}{\text{QBER}}$ we will only detect an error on average. Subsequent block sizes are defined recursively as $B_{i+1} = 2 \cdot B_i$, effectively doubling the block size with each pass. After completion of all iterations, the results are stored and subsequently combined for comprehensive analysis.

3) Summary of the Procedure

In both protocols, our simulation framework allows for the systematic study of parameter variations and their impact on reconciliation efficiency. For the LDPC protocol, the effect of puncturing and shortening is explicitly quantified, while for the Cascade protocol, the influence of block size and iteration count is thoroughly assessed.

V. RESULTS

In this section, we present the experimental results for the LDPC code and Cascade protocol over the two communication channels previously described.

A. LDPC CODE: PUNCTURING ANALYSIS

In this subsection, we investigate the effect of puncturing on the performance of the LDPC code by fixing the rate $R = 0.75$ and the shortening number $s_n = 0$. Puncturing is expected to reduce the code rate and increase efficiency at the expense of error correction capability. We analyze the behavior of the code over both the Binary Symmetric Channel (BSC) and channels with a deterministic number of random errors. The metrics that we will consider aref_1, f_2, and the mean number of protocol iteration.

To analyze the puncturing effect on our metrics of choice we tested the ldpc protocol over different puncturing number: $p_n \in \{50, 100, 150, 200\}$

a: f_1 and f_2 analysis

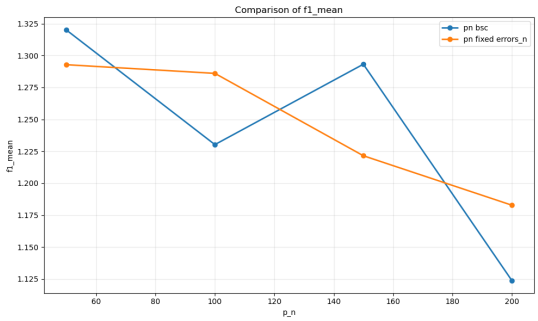


FIGURE 1. Effect of puncturing on LDPC code performance: f_1

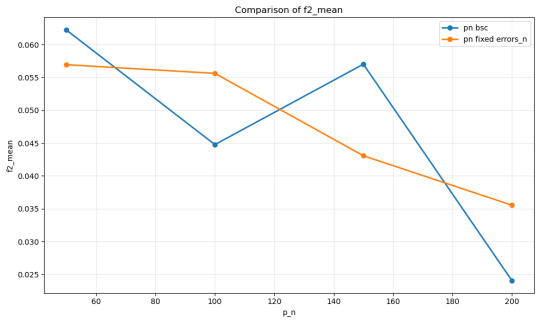


FIGURE 2. Effect of puncturing on LDPC code performance: f_2

b: number of mean iteration

c: Discussion

As expected, the puncturing of the LDPC code improves efficiency with increasing puncturing rate, as observed from Table 1 and Table 2. However, this comes at a trade-off: the minimum number of iterations required to obtain the final key increases with the puncturing level, reflecting the anticipated effects. Also even if we can observe a clear trend in the metrics, for $p_n = 150$ we have a spike in the bsc channel this show how the fluctuation in the errors can lead to noisy result.

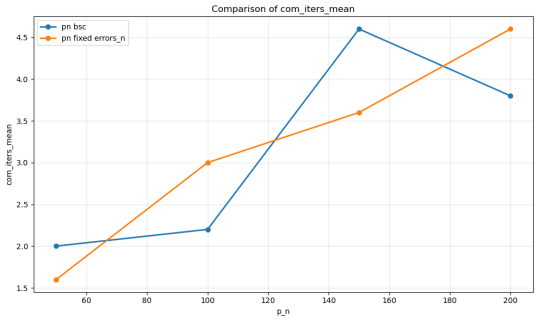


FIGURE 3. Effect of puncturing on LDPC code performance: #iteration

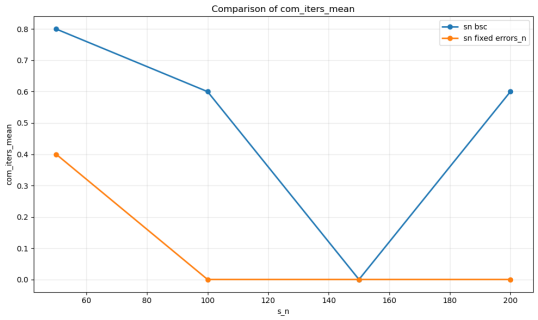


FIGURE 6. Effect of shortening on LDPC code performance: #iteration

B. LDPC CODE: SHORTENING ANALYSIS

We now examine the effect of shortening on the LDPC code performance by fixing the rate $R = 0.75$ and the puncturing number $p_n = 0$. Unlike puncturing, shortening reduces the code length, which can improve error correction at the cost of efficiency.

We tested the protocol with shortening numbers $s_n \in \{50, 100, 150, 200\}$ and analyzed the metrics f_1, f_2, and the mean number of protocol iterations.

a: f_1 and f_2 analysis

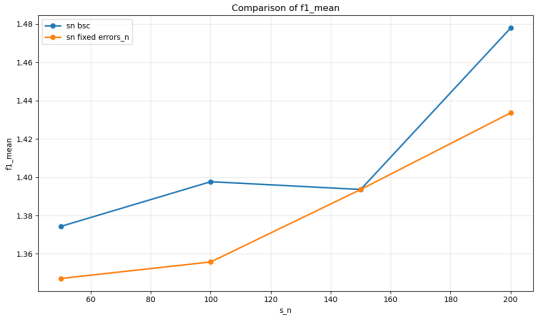


FIGURE 4. Effect of shortening on LDPC code performance: f_1

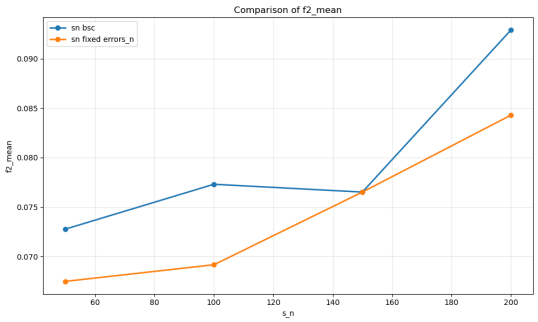


FIGURE 5. Effect of shortening on LDPC code performance: f_2

b: Number of mean iterations

c: Discussion

We can observe how shortening slightly improves error correction metrics form the decreasing trend of the mean iteration number in Table 6, especially for higher s_n, while worsening the efficiency of the protocol. Even in this case we can observe the noisy effect of the BSC channel over the metrics estimations.

C. CASCADE VS. LDPC: COMPARATIVE ANALYSIS

We now compare the LDPC code with auto rate adaptation and the Cascade protocol across the two channels. The goal is to evaluate and contrast f_1, f_2, and the total number of disclosed bits under varying error rates. Notice how we aren't looking at the number of iteration as a metrics to confront the two protocol. This is due to the fact that the two protocol has a cost associated with an iteration that's very different. Infact the simulation time for Cascade lies in the order of seconds while LDPC reach order of minutes.

a: f_1 and f_2 analysis

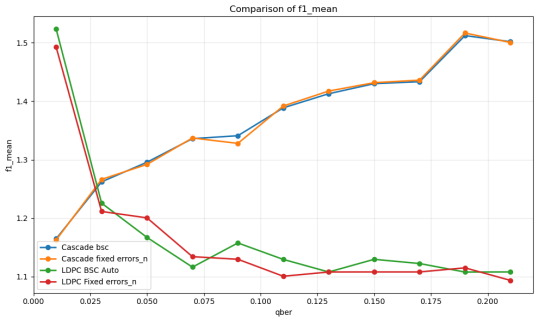


FIGURE 7. Comparison of LDPC and Cascade protocols over BSC and deterministic error channels: f_1.

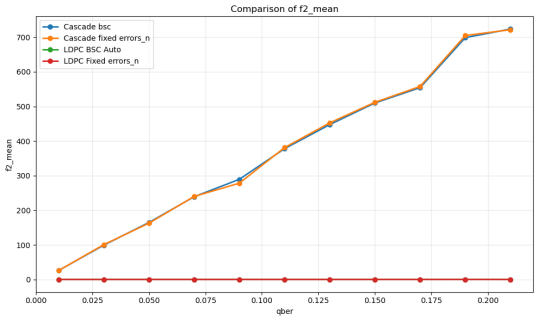


FIGURE 8. Comparison of LDPC and Cascade protocols over BSC and deterministic error channels: f_2.

b: number of bits discosed analysis

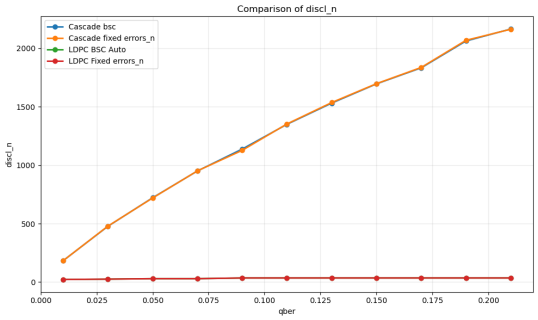


FIGURE 9. Comparison of LDPC and Cascade protocols over BSC and deterministic error channels: total disclosed bits.

c: Discussion

We compared the LDPC and Cascade protocols for information reconciliation, across a range of quantum bit error rates (QBER) from 0.01 to 0.2 with steps of 0.02.

From the analysis, it is observed that at low QBER≤ 0.2, the Cascade protocol initially exhibits better efficiency than LDPC. However, the trends of the two protocols differ as QBER increases: the efficiency of LDPC gradually decreases, whereas Cascade efficiency increases. Consequently, for higher QBER values, Cascade becomes significantly less effective, making it relatively ineffective in such scenarios.

Regarding the number of revealed bits, LDPC consistently maintains values close to zero, while Cascade reveals more bits, particularly for QBER exceeding 0.18, approaching approximately 2000 bits. And since the initial bit stream size is 1944 this means that the Cascade is revealing more bit's than the original. This increase occurs because achieving a perfect success rate requires more iterations in the Cascade protocol, which in turn increases the number of revealed bits. A potential mitigation strategy could be to reduce the number of iterations; however, this would introduce reconciliation errors. To ensure a fair comparison, we decided to maintained a 100% success rate for both protocols, keeping the results consistent. This decision led to an higher number of iteration for Cascade, in particular $n_{iter} = 7$.

Overall, LDPC demonstrates a more stable performance with fewer revealed bits across varying QBER values, while Cascade shows higher efficiency at very low QBER but suffers in terms of efficiency and privacy as the error rate increases. These observations highlight the trade-offs between efficiency, reliability, and information leakage inherent in different reconciliation protocols.

D. EXTRA ANALYSIS

In this section we would show the effect of a reduced number of rounds on the trade-off between a lower number of disclosed bit and the probability of failing the reconciliation. In particular we lowered the number iteration of the cascade protocol from 7 to 4. From the plot of disclosed

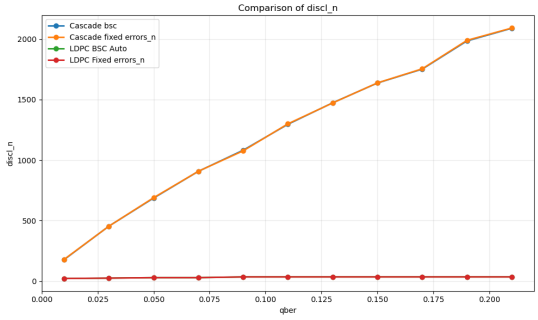


FIGURE 10. Comparison of LDPC and Cascade protocols over BSC and deterministic error channels: discolesd number.

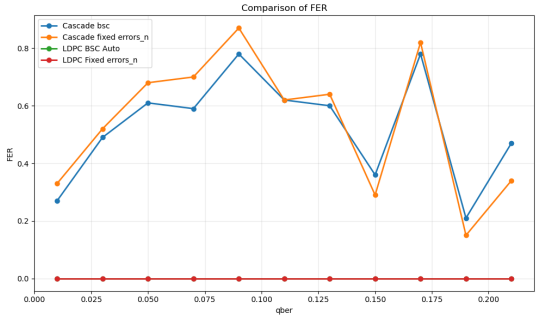


FIGURE 11. Comparison of LDPC and Cascade protocols over BSC and deterministic error channels: FER.

bits, we observe that reducing the number of rounds does not significantly impact the total number of disclosed bits. This behavior can be explained by the fact that the first few rounds, which operate on smaller blocks, detect and correct the majority of the errors. As a result, the last iterations perform only a limited number of searches, since few errors remain to be corrected. However, these additional rounds play an important role in preventing residual errors from going undetected, thereby reducing the probability of failure.

We can therefore conclude that, to achieve a good trade-off between the number of disclosed bits and the residual error rate, the most effective approach is to increase the initial block size.

VI. SIMULATION LIBRARY FOR SYMMETRIC INFORMATION RECONCILIATION IN QKD

In this subsection, we introduce a Python library designed to simulate a symmetric blind reconciliation protocol using LDPC codes. This libraries will be later used to perform the simulation and the evaluation of the protocols metrics

A. LIBRARY STRUCTURE

The repository is composed by three main files:

- **`test_error_correction.py`**: Launches simulations of the symmetric blind reconciliation algorithm. It iterates over user-specified numbers of trials and QBER values, and stores results in `output.txt`.
- **`error_correction_lib.py`**: Core module containing procedures for the reconciliation protocol:
  - Generation of random binary keys.
  - Introduction of errors according to a specified QBER both following a BSC channel and a fixed number of error channel.
  - Selection of an appropriate LDPC code rate from a predefined set, along with determination of shortened and punctured bits.
  - Generation of positions for shortened and punctured bits.
  - Extension of the key with shortened and punctured bits.
  - Syndrome encoding and decoding.
  - Execution of symmetric blind reconciliation between a pair of keys.
  - End-to-end testing of the reconciliation procedure, including key generation, error introduction, and collection of performance statistics.
- **`codes_1944.txt`**: Contains four standard LDPC codes of block length 1944, along with positions for untainted puncturing. The available code rates are $\{\frac{5}{6}, \frac{3}{4}, \frac{2}{3}, \frac{1}{2}\}$.
- **`codes_4000.txt`**: Contains nine LDPC codes of block length 4000, constructed using an improved Progressive Edge-Growth (PEG) algorithm with specific polynomial distributions. The available code rates are $\{0.9, 0.85, \ldots, 0.5\}$.
- **`file_utils.py`**: Provides auxiliary procedures for reading code files and managing data structures.

This storage format enables efficient access during syndrome computation and decoding.

B. EXTENDED TOOLS FOR SIMULATION CONTROL AND DATA MANAGEMENT

To facilitate more controlled and reproducible simulations, we developed an additional set of Python modules that extend the functionality of the base LDPC library. These modules allow fine-grained control over key protocol parameters, as well as convenient data storage, retrieval, and visualization.

1) Description of Additional Modules

VII. CODE IMPLEMENTATION

- **`wrapper_class.py`**: This module rewrites the high-level simulation functions provided by the original LDPC library. Its main purpose is to allow simulations with user-defined fixed numbers of shortened and punctured bits. By wrapping the original logic, it enables systematic exploration of different parameter choices while maintaining compatibility with the original LDPC procedures.
- **`data_saving.py`**: Provides a set of utilities for managing simulation results.
- **`data_class.py`**: Implements a structured class to encapsulate all data required by the reconciliation protocols.
- **`cascade.py`** used to simulate the cascade protocol

VIII. NUMERICAL RESULTS

In this section we are presenting some numerical results obtained by our simulation, in particular we are restricting the case only to BSC channel for simplify the presentation, for the full results details pls consult the /results/ folder in the code repository, were all numerical data are stored in a *csv* format.

a: CASCADE BSC

it.	qber	discl_n	nz_mean	f1_mean	f2_mean	success_rate
0	0.01	182.63	1944.0	1.162790	25.568144	0.74
1	0.03	478.53	1944.0	1.266295	100.632228	0.88
2	0.05	719.32	1944.0	1.291985	162.564315	0.92
3	0.07	951.17	1944.0	1.337123	239.814423	0.90
4	0.09	1126.71	1944.0	1.327889	278.212676	0.95
5	0.11	1352.62	1944.0	1.391818	380.783377	0.99
6	0.13	1535.99	1944.0	1.417410	452.330168	0.98
7	0.15	1697.37	1944.0	1.431740	511.840448	1.00
8	0.17	1836.11	1944.0	1.436056	557.531910	0.99
9	0.19	2068.38	1944.0	1.516785	704.719482	1.00

b: LDPC BSC

it.	com_iters_mean	qber	f1_mean	f2_mean	R	s_n	p_n	p_n_max	k_n	discl_n	FER
0	3.4	0.01	1.523382	0.042286	0.8333	0	182	152	1762	22	0.0
1	3.2	0.03	1.225634	0.043861	0.7500	0	135	227	1809	25	0.0
2	3.0	0.05	1.166966	0.047818	0.6667	0	128	302	1816	29	0.0
3	2.6	0.07	1.116171	0.042510	0.6667	173	0	302	1771	29	0.0
4	3.4	0.09	1.157515	0.068751	0.5000	0	220	439	1724	35	0.0
5	3.6	0.11	1.129372	0.064675	0.5000	0	1	439	1943	35	0.0
6	3.0	0.13	1.107828	0.060108	0.5000	200	0	439	1744	35	0.0
7	3.6	0.15	1.129530	0.078993	0.5000	350	0	439	1594	35	0.0
8	3.4	0.17	1.122327	0.080455	0.5000	466	0	439	1478	35	0.0
9	3.0	0.19	1.107752	0.075585	0.5000	558	0	439	1386	35	0.0

c: LDPC PUNCTURING BSC

it.	com_iters_mean	qber	f1_mean	f2_mean	R	s_n	p_n	p_n_max	k_n	discl_n	FER
0	2.0	0.03	1.320013	0.062208	0.75	0	50	227	1894	25	0.0
1	2.2	0.03	1.230268	0.044762	0.75	0	100	227	1844	25	0.0
2	4.6	0.03	1.293231	0.057002	0.75	0	150	227	1794	25	0.0
3	3.8	0.03	1.123829	0.024071	0.75	0	200	227	1744	25	0.0

d: LDPC SHORTENING BSC

it.	com_iters_mean	qber	f1_mean	f2_mean	R	s_n	p_n	p_n_max	k_n	discl_n	FER
0	0.8	0.03	1.374335	0.072768	0.75	50	0	227	1894	25	0.0
1	0.6	0.03	1.397651	0.077300	0.75	100	0	227	1844	25	0.0
2	0.0	0.03	1.393592	0.076511	0.75	150	0	227	1794	25	0.0
3	0.6	0.03	1.477792	0.092879	0.75	200	0	227	1744	25	0.0